

# C++ Notes

Value / Reference / Move / Forward

Adam HAMOUCH

May 3, 2026

## 1. Définition

**lvalue** : objet avec une adresse persistante (variable nommée)

**rvalue** : valeur temporaire (littéral, résultat d'expression)

**Référence** : alias d'une variable existante

- **T&** : référence vers lvalue
- **T&&** : référence vers rvalue (temporaire)

Une référence :

- doit être initialisée
- ne peut pas être rebind

**Important** : une variable nommée est toujours une **lvalue**

## 2. Problème résolu

- Éviter les copies inutiles
- Permettre le transfert efficace de ressources
- Distinguer objets persistants vs temporaires
- Préserver la nature d'un objet dans du code générique

### 3. Fonctionnement

#### Déduction de type :

- `T param` → copie (perd const et référence)
- `T& param` → garde const
- `T&& param` → forwarding reference

#### Règle (reference collapsing) :

- lvalue → devient `T&`
- rvalue → reste `T&&`

#### `auto` :

- `auto` → copie
- `auto&` → référence
- `auto&&` → dépend (comme `T&&`)

#### `std::move` :

- cast en rvalue
- ne déplace pas directement
- permet d'appeler le move constructor

#### `std::forward` :

- préserve lvalue / rvalue
- utilisé avec `T&&`

### 4. Exemple minimal

```
int a = 10;

int& lref = a;
int&& rref = 10;

auto x = a;           // copie
auto& y = a;          // ref
auto&& z = a;          // int&
auto&& w = 10;         // int&&

// move
std::vector<int> v = {1,2,3};
std::vector<int> v2 = std::move(v);
```

```
// forward
template<typename T>
void wrapper(T&& arg) {
    func(std::forward<T>(arg));
}
```

## 5. Pièges classiques

- T&& n'est pas toujours une rvalue reference
- une variable nommée est toujours une lvalue
- auto peut copier silencieusement
- `std::move(const T)` → copie
- oublier `std::forward` casse le move
- utiliser `std::move` trop tôt invalide un objet

## 6. Coût et performance

- Copy →  $O(n)$ , duplication mémoire
- Move →  $O(1)$ , transfert de pointeur
- Référence → aucun coût

Critique pour :

- `std::vector`
- `std::string`
- Mesh / textures / buffers

## 7. Quand utiliser / éviter

### 7.1 Utiliser

- `const T&` pour éviter copies
- `std::move` pour transférer des ressources
- `std::forward` en code générique

### 7.2 Éviter

- copies implicites

- move sur objets encore utilisés
- move sur const